

SistRUGBY-SLTC

Sistema de Gestión de Partidos y Estadísticas de Rugby
Santiago Lawn Tennis Club · Santiago del Estero

Trabajo Práctico N.º 4 (AP4)

Versión final integradora — Entrega incremental sobre AP1, AP2 y AP3
Fase de Transición del Proceso Unificado de Desarrollo

Alumno: Fonzo, Ignacio

Licenciatura en Informática — Universidad Siglo 21
Seminario de Práctica — Módulo 4 · Junio de 2026

Repositorio: <https://github.com/nachof9/SistRUGBY-SLTC>

1. Introducción

El presente documento corresponde al Trabajo Práctico N.º 4 (AP4) de la materia Seminario de Práctica de la Licenciatura en Informática de la Universidad Siglo 21, modalidad a distancia. Constituye la entrega final integradora del proyecto SistRUGBY-SLTC. Por su carácter iterativo e incremental —propio del Proceso Unificado de Desarrollo (PUD)—, este trabajo retoma e incluye el contenido de AP1 (análisis), AP2 (diseño y base de datos) y AP3 (prototipo POO), incorpora las correcciones de la cátedra y agrega la contribución específica de AP4 correspondiente a la fase de Transición.

La novedad central de AP4 es que el prototipo deja de operar exclusivamente en memoria y queda funcional con conexión real a una base de datos MySQL, tal como solicitó la retroalimentación de la entrega anterior. Las contribuciones de esta entrega son:

- Persistencia y consulta en MySQL vía JDBC: establecer conexiones, consultar, actualizar registros y presentar resultados.
- Selección y justificación del patrón de diseño (DAO), materializado en una interfaz Java.
- Manejo de excepciones en la interacción con MySQL.
- Clases abstractas (Persona, EventoPartido) e interfaz (JugadorDAOInterface).
- Utilización complementaria de arreglos y de la clase ArrayList.
- Empleo de archivos para guardar información (generación de reportes).

El sistema SistRUGBY-SLTC centraliza la gestión operativa de la sección de rugby del Santiago Lawn Tennis Club (SLTC) de Santiago del Estero, reemplazando los registros manuales y planillas no integradas, y proveyendo al cuerpo técnico información oportuna y confiable para la toma de decisiones.

2. Resumen de AP1 — Análisis y definición del proyecto

2.1 Justificación y modelo de negocio

La sección de rugby del SLTC enfrenta la ausencia de un sistema centralizado para gestionar jugadores, registrar partidos y generar estadísticas. El cuerpo técnico y el secretario deportivo registran los datos en papel o planillas Excel individuales sin integración. Los problemas identificados son: pérdida o inconsistencia de datos históricos, imposibilidad de generar reportes ágiles, dificultad de seguimiento del rendimiento individual, tiempo administrativo excesivo y riesgo de pérdida por soportes físicos sin respaldo.

Desde la perspectiva del modelo de negocio, el cuerpo técnico y la secretaría son los clientes internos; su propuesta de valor es disponer de información confiable para decisiones deportivas (convocatorias, planificación, seguimiento). Las actividades clave son el registro del padrón, la carga de partidos y eventos y la generación de estadísticas; el recurso clave, una base de datos centralizada. El proyecto se justifica porque elimina los costos ocultos del trabajo manual (retrabajo, errores, pérdida de datos) y habilita un activo reutilizable: el histórico deportivo del club.

2.2 Alcance

El sistema cubre las divisiones juveniles M15, M16, M17, M18 y M19, y el plantel superior (Pre-Intermedia, Intermedia y Primera). Quedan fuera del alcance las categorías infantiles, la gestión financiera, la transmisión en vivo, la integración con redes sociales y otras disciplinas. El prototipo es una aplicación de escritorio con persistencia en MySQL; no incluye versión web ni móvil.

2.3 Requerimientos funcionales

ID	Actor	Descripción	Prior.
RF01	Secretario	Registrar, modificar, dar de baja y consultar jugadores.	Alta
RF02	Entrenador	Registrar partidos (fecha, rival, resultado, sede, categoría).	Alta
RF03	Entrenador	Asociar jugadores a un partido (titular/suplente).	Alta
RF04	Entrenador	Registrar eventos: try, conversión, penal, drop, tarjetas, sustituciones.	Alta
RF05	Entrenador	Generar estadísticas por jugador.	Alta
RF06	Entren./Secr.	Generar reporte por partido.	Media
RF07	Administrador	Administrar las divisiones.	Media
RF08	Secretario	Mantener el listado de clubes rivales.	Media
RF09	Todos	Acceso con usuario y contraseña con roles diferenciados.	Alta
RF10	Sistema	Organizar partidos y estadísticas por temporada.	Media

2.4 Requerimientos no funcionales

ID	Categoría	Descripción
RNF01	Seguridad	Autenticación; contraseñas con hash robusto (PBKDF2).
RNF02	Rendimiento	Consultas y reportes en menos de 3 s para hasta 500 partidos.
RNF03	Usabilidad	Interfaz intuitiva, sin capacitación técnica especializada.
RNF04	Confiabilidad	Integridad referencial por claves foráneas.
RNF05	Portabilidad	Ejecutable en Windows 10/11 con JRE 17+.
RNF06	Mantenibilidad	Código POO en capas (presentación, negocio, persistencia).
RNF07	Disponibilidad	Operativo en horario del club (7:00–23:00 hs).

2.5 Procesos de negocio actuales (As-Is)

Se identifican tres procesos críticos, hoy ejecutados manualmente: inscripción de jugadores, organización y registro de partidos, y seguimiento estadístico.

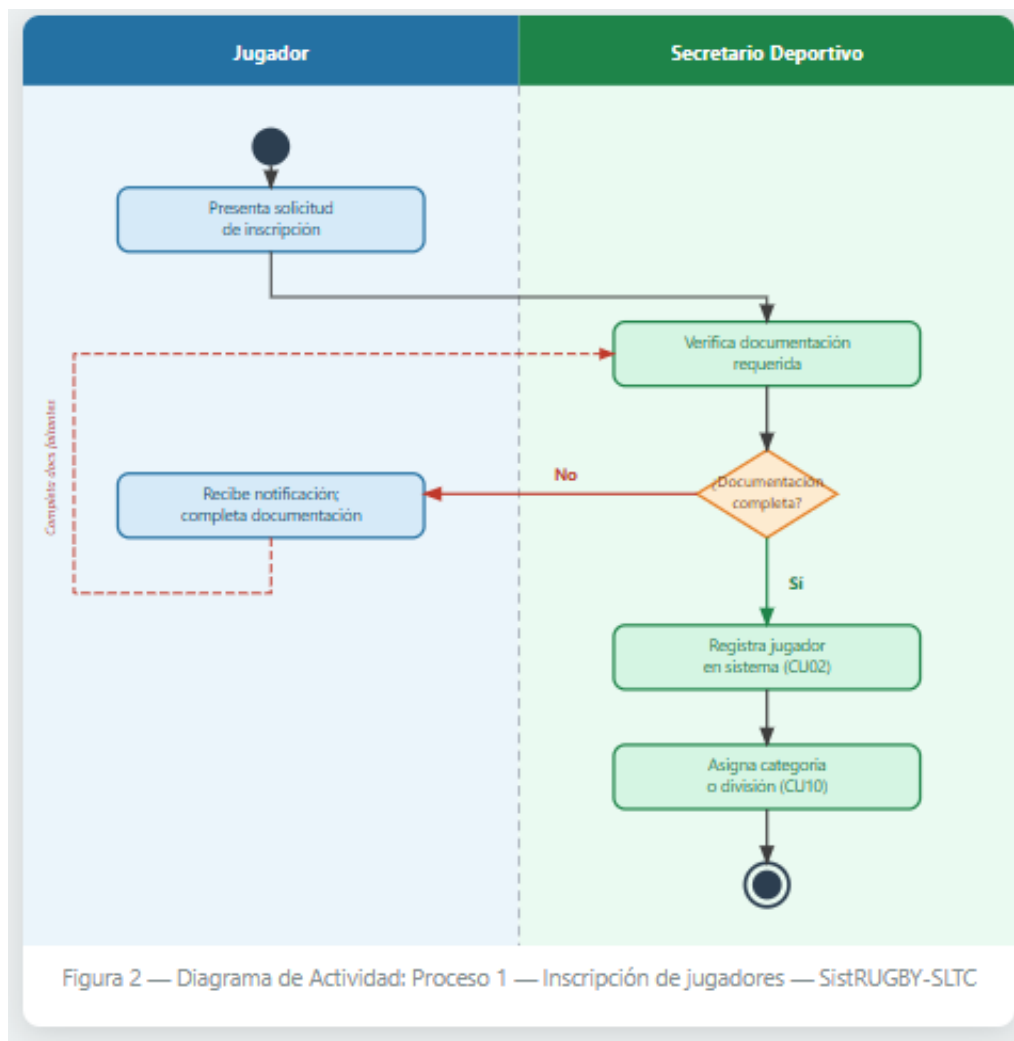


Figura 1 — Diagrama de actividad: inscripción de jugadores.

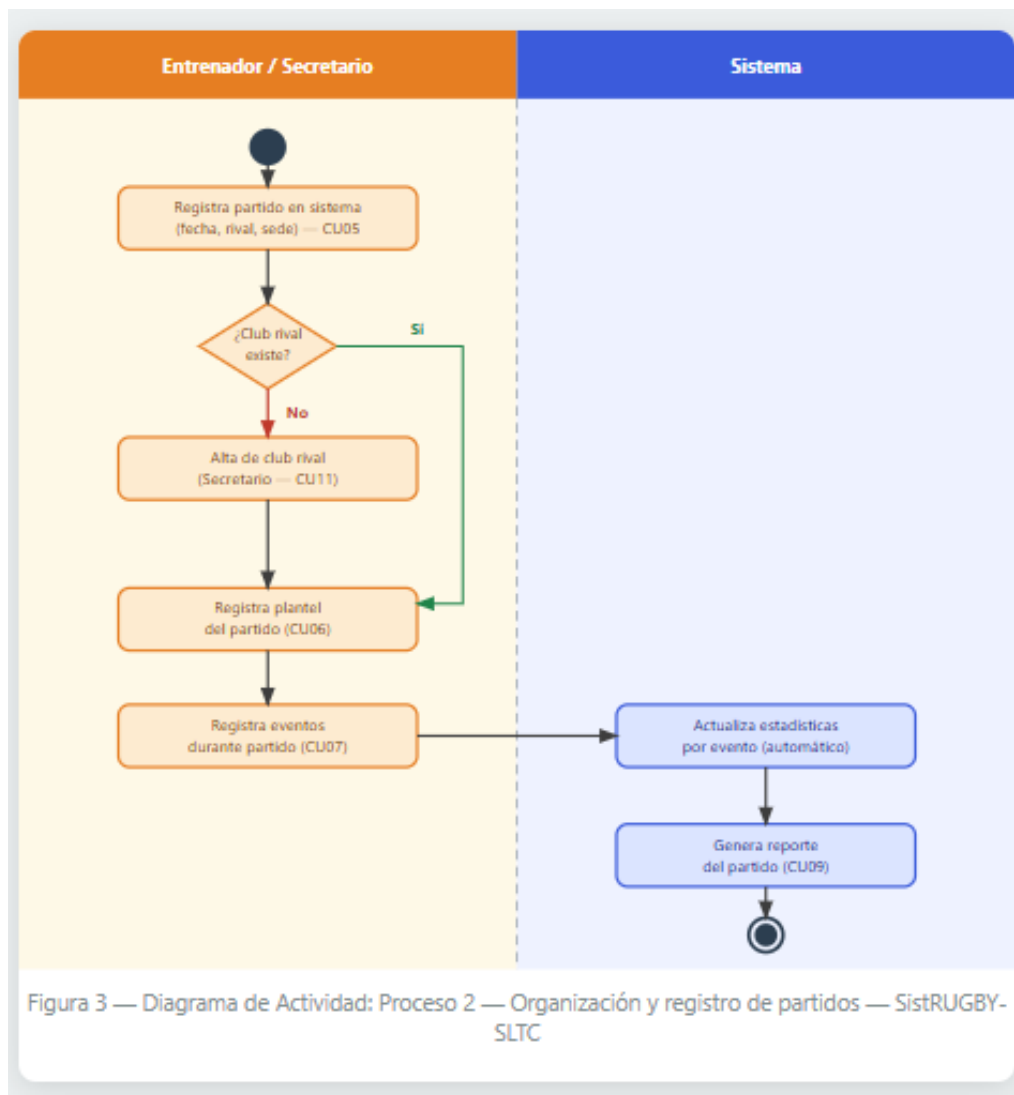


Figura 2 — Diagrama de actividad: organización y registro de partidos.

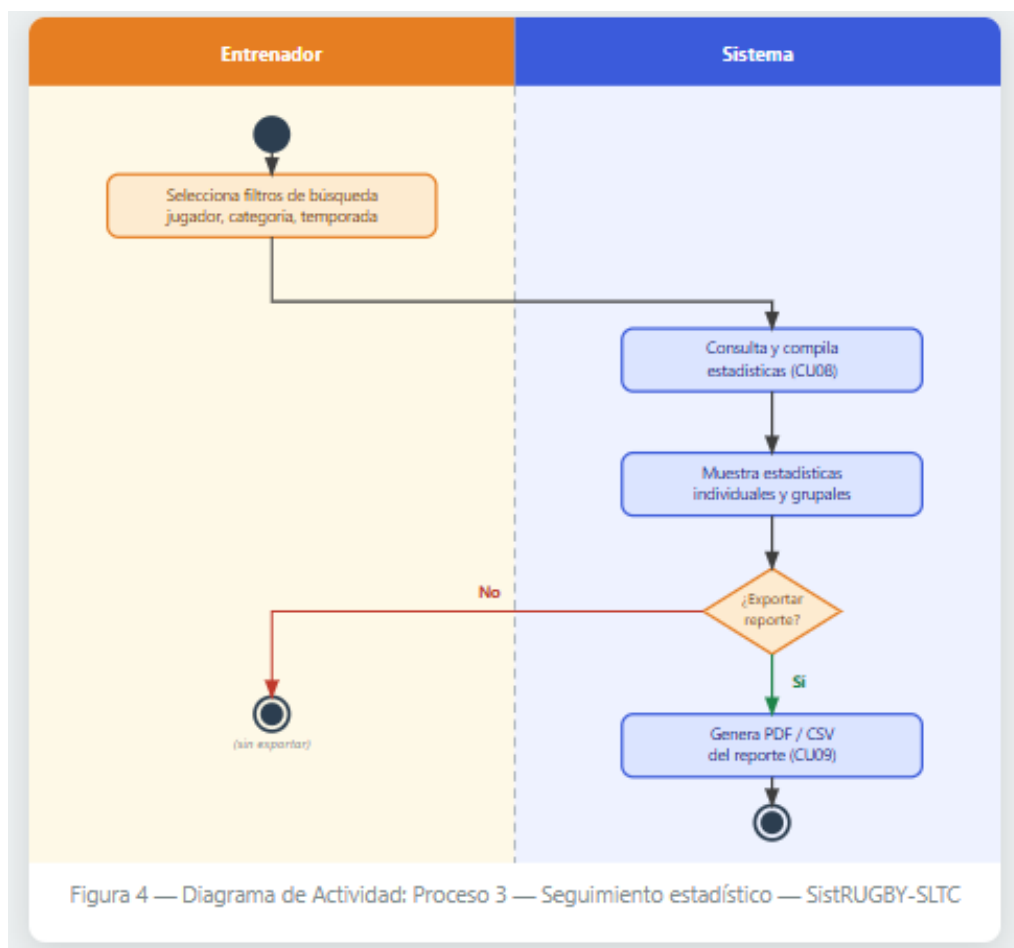


Figura 3 — Diagrama de actividad: seguimiento estadístico.

2.6 Casos de uso

Se definieron doce casos de uso en cuatro grupos funcionales. Todos los actores participan en CU01 (Iniciar sesión).

CU	Nombre	Actor	Grupo
CU01	Iniciar sesión	Todos	Acceso
CU02	Registrar jugador	Secretario	Gestión de jugadores
CU03	Modificar jugador	Secretario	Gestión de jugadores
CU04	Dar de baja jugador	Secretario	Gestión de jugadores
CU05	Registrar partido	Entrenador	Gestión de partidos
CU06	Registrar plantel	Entrenador	Gestión de partidos
CU07	Registrar evento	Entrenador	Gestión de partidos
CU08	Consultar estadísticas	Entrenador	Estadísticas y reportes
CU09	Generar reporte	Entren./Secr.	Estadísticas y reportes
CU10	Administrar categorías	Administrador	Configuración
CU11	Administrar clubes	Secretario	Configuración
CU12	Gestionar usuarios	Administrador	Configuración

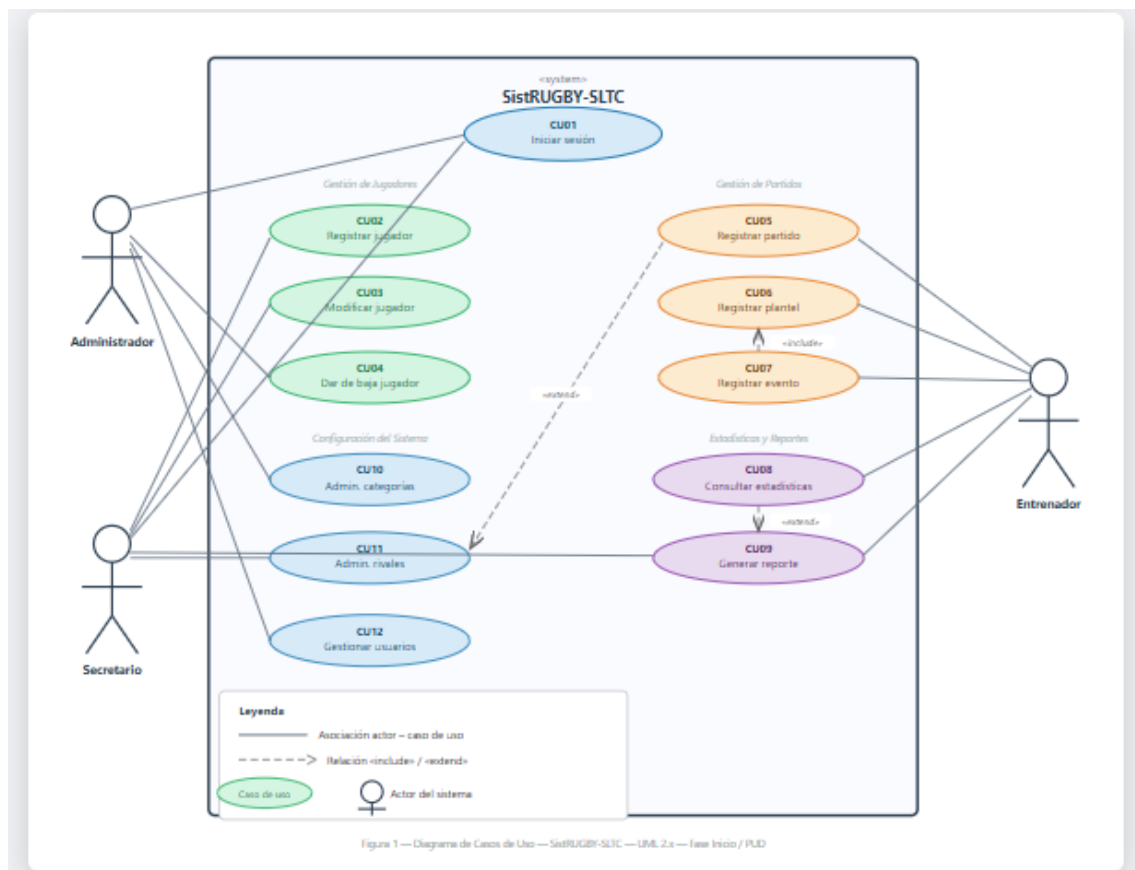


Figura 4 — Diagrama de casos de uso (UML 2.x).

2.7 Análisis de riesgos

En la fase de Inicio del PUD se identificaron diez riesgos. Los críticos: modificación de requisitos durante la construcción (mitigado con congelamiento de alcance en Elaboración), carácter unipersonal del proyecto (mitigado con documentación en GitHub e hitos semanales) e inconsistencia de las planillas Excel originales (mitigado con auditoría y limpieza previa a la migración).

3. Resumen de AP2 — Diseño y base de datos

3.1 Fase de Elaboración: análisis

La Fase de Elaboración estabiliza la arquitectura, refina los casos de uso de mayor riesgo e identifica los elementos del dominio. El modelo de dominio identifica ocho entidades: Jugador, Partido, EventoPartido, PlantelPartido, Club, Categoría, Temporada y Usuario.

Se elaboraron diagramas de secuencia para los casos de uso de mayor criticidad arquitectónica — CU01 (autenticación) y CU05 (registro de partido) —, que muestran la colaboración entre las tres capas (presentación, negocio y persistencia) y la interacción con MySQL vía JDBC.

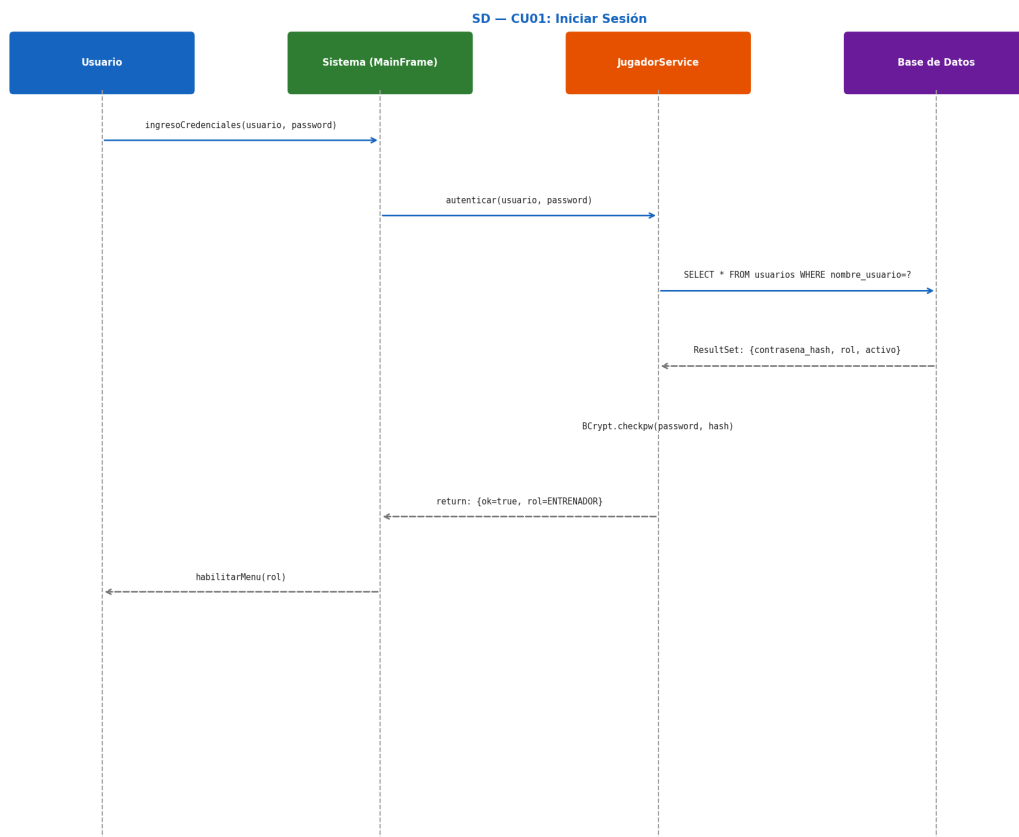


Figura 5 — Diagrama de secuencia CU01 (Iniciar sesión).

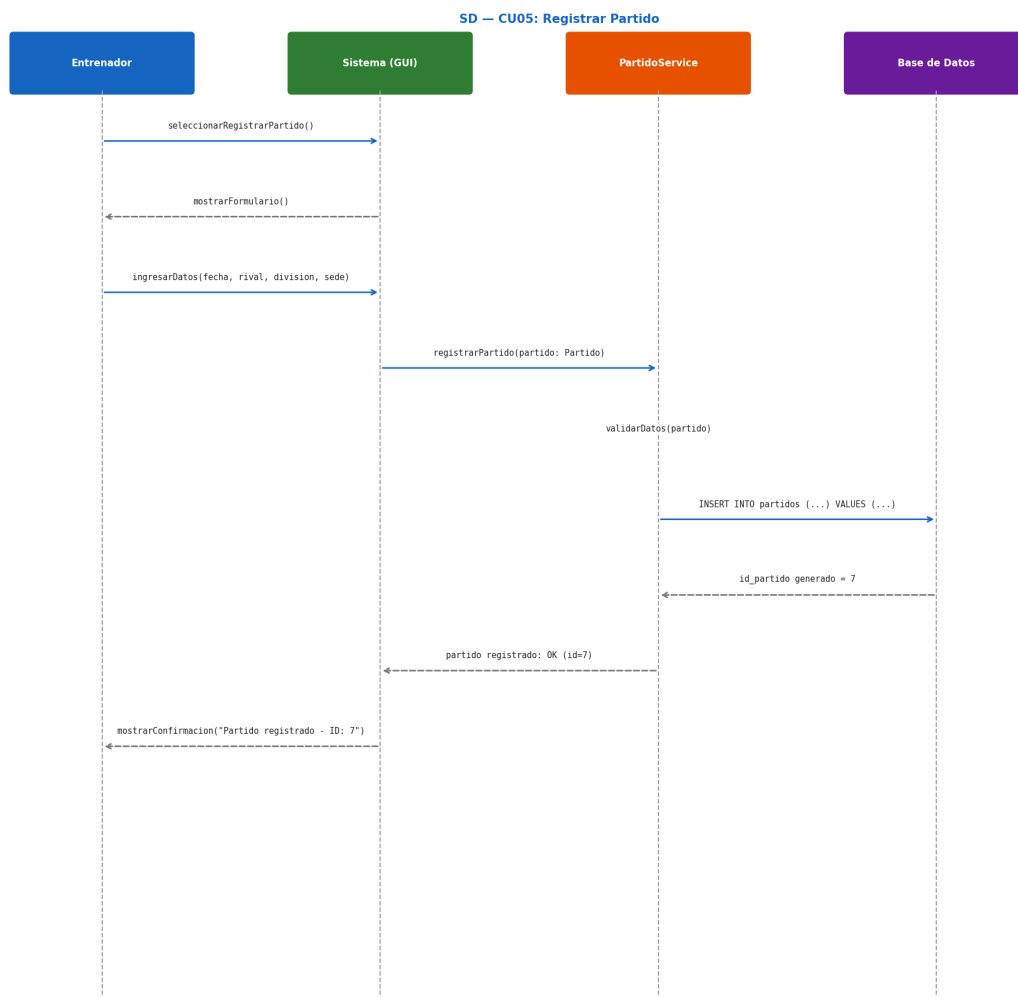


Figura 6 — Diagrama de secuencia CU05 (Registrar partido).

3.2 Fase de diseño

SistRUGBY-SLTC adopta una arquitectura en tres capas (Layered Architecture): Presentación (menú por consola; en AP2 también Swing), Negocio (clases Service con la lógica de aplicación y validaciones) y Persistencia (clases DAO que acceden a MySQL vía JDBC, con ConexionBD como Singleton). El diagrama de clases detalla entidades POJO, servicios, DAOs e interfaz.

```
classDiagram
    class Usuario {
        -id : int
        -nombreusuario : String
        -contrasenaa : String
        -rol : Rol
        -activo : boolean
        +autenticar(pass:String) : boolean
        +cambiarRol() : boolean
    }
    class Division {
        -id : int
        -nombre : String
        -tipo : TipoDivision
        +getJugadores() : List<Jugador>
    }
    class ClubRival {
        -id : int
        -nombre : String
        -unionPart : String
        +getPartidos() : List<Partido>
    }
    class Jugador {
        -id : int
        -nombre : String
        -apellido : String
        -dni : String
        -fechaNac : LocalDate
        -posicion : String
        -estado : EstadoJugador
        +getNombreCompleto() : String
        +estaActivo() : boolean
    }
    class Partido {
        -id : int
        -fecha : LocalDate
        -sede : Sede
        -puntosLocal : int
        -puntosVisit : int
        -temporada : int
        -estado : EstadoPartido
        +getEventos() : List<EventoPartido>
        +getPlantel() : List<PlantelPartido>
    }
    class PlantelPartido {
        -id : int
        -idPartido : int
        -idJugador : int
        -rol : RolPartido
    }
    class EventoPartido {
        -id : int
        -idPartido : int
        -idJugador : int
        -tipoEvento : TipoEvento
        -minuto : int
        +getPhotosAportados() : List
    }
    class Rol {
        <<enumeration>>
        ADMINISTRADOR
        ENTRENADOR
        SECRETARIO
    }
    class TipoEvento {
        <<enumeration>>
        TRY
        CONVERSION
        PENAL
        DROP
        TARJETA_AMARILLA
        TARJETA_ROJA
        SUSTITUCION
    }
    class TipoDivision {
        <<enumeration>>
        JUVENIL
        PLANTEL_SUPERIOR
    }
    Usuario "1" -- "0..*" Division
    Division "1" -- "0..*" ClubRival
    ClubRival "1" -- "0..*" Rol
    Division "1" -- "0..*" Partido
    Jugador "1" -- "0..*" Partido
    Partido "1" -- "0..*" PlantelPartido
    Partido "1" -- "0..*" EventoPartido
    PlantelPartido "1" -- "0..*" EventoPartido
```

Figura 7 — Diagrama de clases (análisis).

[illegible]

Figura 8 — Diagrama de clases (diseño).

Diagrama de Componentes — SistrUGBY-SLTC

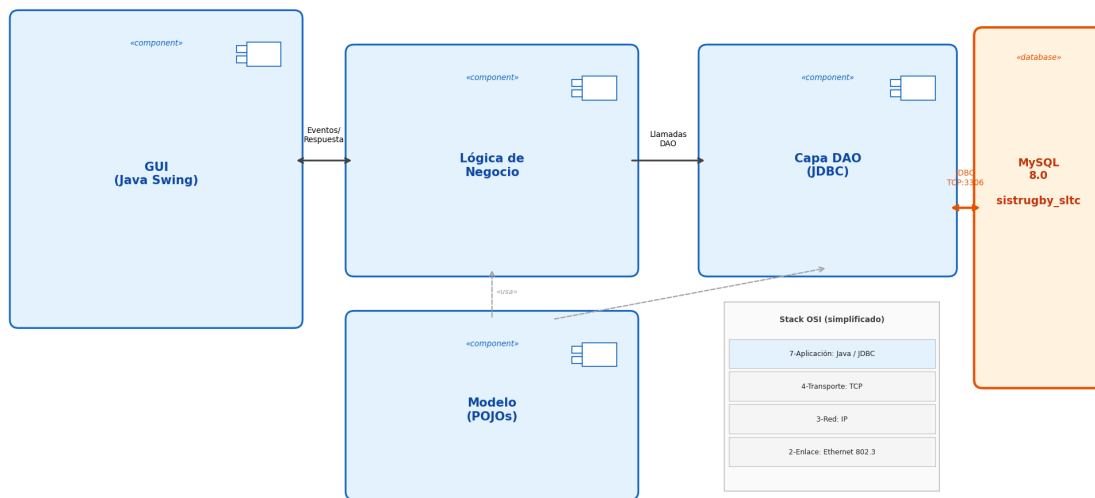


Figura 9 — Diagrama de componentes (UML 2.x).

Diagrama de Despliegue — SistrUGBY-SLTC

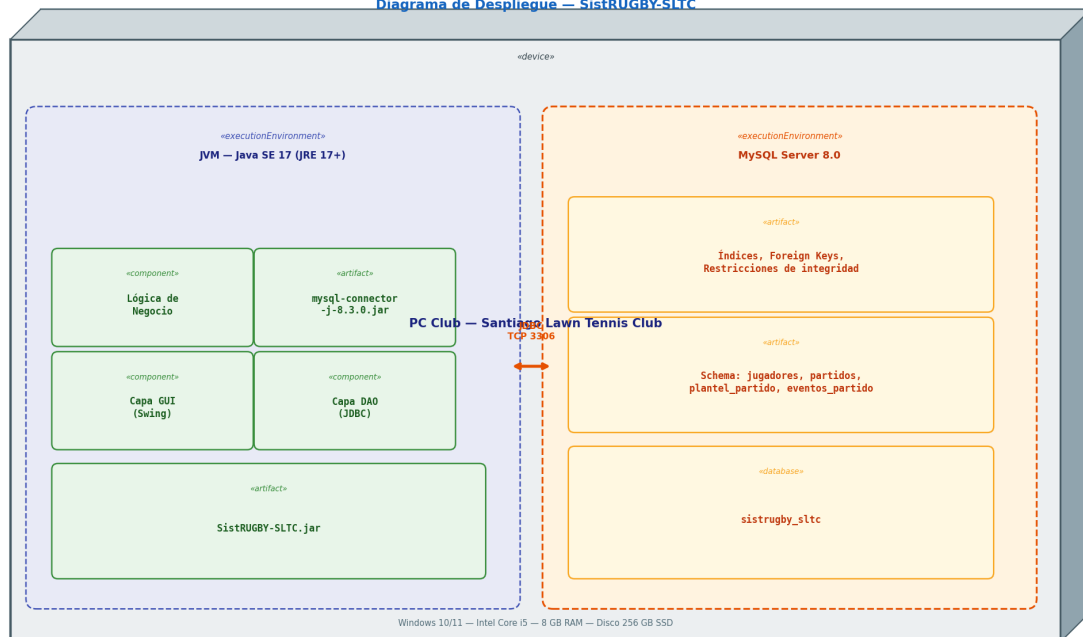


Figura 10 — Diagrama de despliegue (UML 2.x).

3.3 Base de datos

El modelo se diseñó hasta la Tercera Forma Normal (3FN). 1FN: atributos atómicos (cada evento es una fila; los roles son ENUM). 2FN: claves primarias simples por tabla. 3FN: ningún atributo no clave depende de otro no clave. Las relaciones se implementan con claves foráneas e integridad referencial (ON DELETE RESTRICT/CASCADE, ON UPDATE CASCADE). El esquema completo (sql/schema.sql) define ocho tablas: usuarios, categorías, clubes, temporadas, jugadores, partidos, partido_plantel y eventos_partido.

```

CREATE TABLE jugadores (
  id_jugador      INT          AUTO_INCREMENT PRIMARY KEY,
  nombre          VARCHAR(80)  NOT NULL,
  apellido        VARCHAR(80)  NOT NULL,
  dni             VARCHAR(10)  NOT NULL UNIQUE,
  fecha_nacimiento DATE        NOT NULL,
  posicion        VARCHAR(30),
  id_categoria    INT          NOT NULL,

```

```

estado          ENUM('activo','inactivo') NOT NULL DEFAULT 'activo',
fecha_alta      DATE NOT NULL DEFAULT (CURDATE()),
fecha_baja      DATE,
CONSTRAINT fk_jug_cat FOREIGN KEY (id_categoria)
REFERENCES categorias (id_categoria) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;

CREATE TABLE eventos_partido (
id_evento      INT AUTO_INCREMENT PRIMARY KEY,
id_partido     INT NOT NULL, id_jugador INT NOT NULL,
tipo_evento    ENUM('TRY','CONVERSION','PENAL','DROP',
'TARJETA_AMARILLA','TARJETA_ROJA','SUSTITUCION') NOT NULL,
minuto         INT, descripcion VARCHAR(200),
CONSTRAINT fk_ev_partido FOREIGN KEY (id_partido)
REFERENCES partidos (id_partido) ON UPDATE CASCADE ON DELETE CASCADE,
CONSTRAINT fk_ev_jugador FOREIGN KEY (id_jugador)
REFERENCES jugadores (id_jugador) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;

```

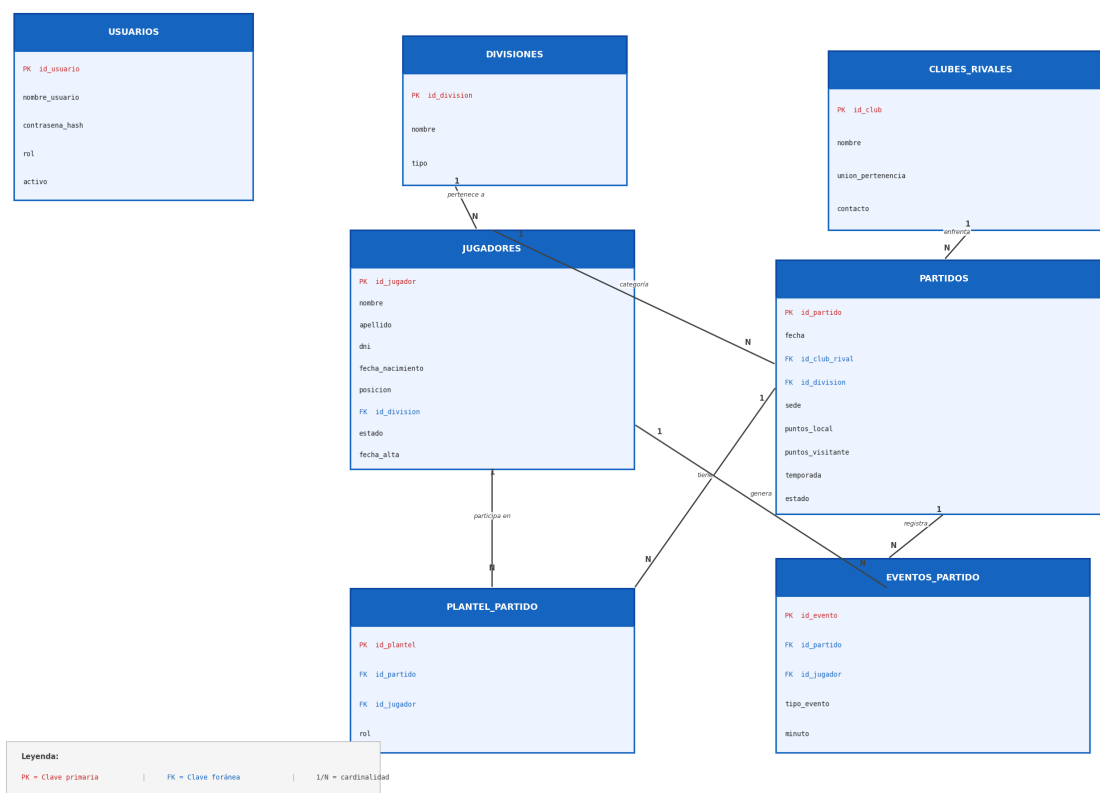


Figura 11 — Diagrama Entidad-Relación (MySQL 8.0).

4. AP3 — Implementación del prototipo en Java (POO)

Esta sección corresponde a la fase de Construcción del PUD. El prototipo en Java SE aplica estrictamente los cuatro pilares de la POO: encapsulamiento, herencia, polimorfismo y abstracción. Compila sin errores y se ejecuta incluso sin MySQL, gracias a un mecanismo de modo dual.

4.1 Stack tecnológico

Componente	Tecnología	Justificación
Lenguaje	Java SE 17	LTS; sintaxis moderna (var, switch expr., records).
IDE	IntelliJ IDEA	Análisis estático y soporte JDBC.
Persistencia	MySQL 8.0 (JDBC)	Driver mysql-connector-j 8.3.0.
Hash	PBKDF2-HMAC-SHA256	NIST SP 800-132; en el JDK, sin dependencias.

Componente	Tecnología	Justificación
Versionado	Git + GitHub	Repositorio público nachof9/SistRUGBY-SLTC.
Metodología: PUD (RUP)		Inicio/Elaboración/Construcción/Transición.

4.2 Estructura del proyecto

El código se organiza en seis paquetes alineados con las capas arquitectónicas, más tres auxiliares (modelo, utilidades, excepciones):

```
src/main/java/com/sltc/sistrugby/
|- Main.java                      Punto de entrada
|- modelo/
|  |- Persona.java                Clase ABSTRACTA (abstracción)
|  |- Jugador.java / Usuario.java extends Persona (herencia)
|  |- Partido, PlantelPartido, Categoria, Club, Temporada
|  `-- eventos/
|     |- EventoPartido.java       Clase ABSTRACTA + Factory Method
|     `-- Try, Conversion, Penal, Drop, TarjetaAmarilla,
|         TarjetaRoja, Sustitucion POLIMORFISMO: calcularPuntos()
|- excepciones/ DniDuplicado, JugadorNoEncontrado, CredencialesInvalidas, DatosInvalidos
|- persistencia/ ConexionBD (Singleton), RepositorioMemoria,
|                 JugadorDAOInterface, JugadorDAO, UsuarioDAO, PartidoDAO,
|                 EventoDAO, CategoriaDAO, ClubDAO, TemporadaDAO (patrón DAO)
|- negocio/      UsuarioService, JugadorService, PartidoService,
|                 EventoService, EstadisticaService, ReporteService
|- util/         HashUtil (PBKDF2), ValidadorDNI, OrdenadorJugadores (QuickSort),
|                 BuscadorJugadores (busqueda binaria)
`-- presentacion/ MenuConsola, DatosInicialesSeeder
```

4.3 Pilares de la Programación Orientada a Objetos

Abstracción. Dos clases abstractas representan el dominio focalizando lo esencial: *Persona* (raíz de *Jugador/Usuario*) y *EventoPartido* (raíz de los siete tipos de evento). Declaran métodos abstractos que definen un contrato sin implementación.

```
public abstract class Persona {
    protected int id; protected String nombre, apellido, dni;
    protected LocalDate fechaNacimiento;
    public String getNombreCompleto() { return apellido + ", " + nombre; }
    public abstract String descripcionCorta(); // contrato
}

public abstract class EventoPartido {
    public enum Tipo { TRY, CONVERSION, PENAL, DROP,
                     TARJETA_AMARILLA, TARJETA_ROJA, SUSTITUCION }
    public abstract int calcularPuntos();
    public abstract Tipo getTipo();
}
```

Herencia. *Jugador* y *Usuario* extienden *Persona* (reutilizan *id*, *nombre*, *apellido*, *dni*, *fechaNacimiento*). Las siete clases de evento extienden *EventoPartido*.

```
public class Jugador extends Persona {
    public enum Estado { ACTIVO, INACTIVO }
    private String posicion; private int idCategoria; private Estado estado;
    public Jugador(String nombre, String apellido, String dni,
                   LocalDate fn, String posicion, int idCategoria) {
        super(0, nombre, apellido, dni, fn); // constructor padre
        this.posicion = posicion; this.idCategoria = idCategoria;
        this.estado = Estado.ACTIVO;
    }
}
```

Encapsulamiento. Todos los atributos son *private* (o *protected* para herencia controlada); el acceso es por *getters/setters*. Los *enums* internos (*Estado*, *Rol*, *Tipo*, *Sede*, *Condicion*) tipifican los valores válidos en lugar de *Strings* sueltos.

Polimorfismo. Las siete subclases sobrescriben `calcularPuntos()`. El código cliente opera sobre el tipo base `EventoPartido` y delega en cada objeto el cálculo correcto, sin `instanceof`:

```
public class Try          extends EventoPartido { public int calcularPuntos(){return 5;} }
public class Conversion  extends EventoPartido { public int calcularPuntos(){return 2;} }
public class Penal       extends EventoPartido { public int calcularPuntos(){return 3;} }

// Recalculo polimorfo del marcador
int total = eventos.stream().mapToInt(EventoPartido::calcularPuntos).sum();
```

Además, `EventoPartido` incluye una fábrica estática (Factory Method) que, a partir del enum `Tipo`, devuelve la instancia concreta, de modo que el cliente no conoce los constructores de cada subclase.

4.4 Manejo de excepciones

El prototipo define cuatro excepciones verificadas de dominio (`DniDuplicadoException`, `JugadorNoEncontradoException`, `CredencialesInvalidasException`, `DatosInvalidosException`) y las distingue de las técnicas (`SQLException`, `NumberFormatException`, `DateTimeParseException`). Se aplica multi-catch y orden de captura de la más específica a la más genérica:

```
try {
    Jugador j = jugadorSvc.registrar(nom, ape, dni, fn, pos, cat);
    System.out.println("OK. " + j.descripcionCorta());
} catch (DatosInvalidosException | DniDuplicadoException e) { // negocio
    System.out.println("Error de validacion: " + e.getMessage());
} catch (DateTimeParseException e) {
    System.out.println("Formato de fecha invalido (YYYY-MM-DD).");
} catch (Exception e) { // catch-all defensivo
    System.out.println("Error inesperado: " + e.getMessage());
}
```

4.5 Sintaxis, tipos y estructuras de control

Se emplean `if/else` (validaciones), `switch/case` (factory de eventos, menú), `for` clásico (`QuickSort`), `for-each` (recorridos polimórficos), `while` (menú y reintentos de login), `do/while` (lectura robusta) y `try-with-resources` (cierre de `PreparedStatement/ResultSet`). Tipos: primitivos (`int`, `boolean`), `String`, `java.time.LocalDate`, enums tipados y colecciones genéricas (`List`, `LinkedList`, `Deque`, `Queue`, `HashMap`).

4.6 Constructores

Cada entidad expone constructor por defecto, constructor de alta (sin id, asignado por la BD) y constructor completo (mapeo desde `ResultSet`).

4.7 Menú de selección

`MenuConsola` implementa un menú por consola con `while` + `switch` (10 opciones), elegido por ser más auditable para el corrector; la GUI Swing del AP2 se conectaría a los mismos Services sin cambiar la lógica. El bucle principal es la combinación idiomática `while` + `switch`:

```
boolean salir = false;
while (!salir) {
    mostrarMenuPrincipal();
    int opcion = leerEntero("Opcion: ");
    switch (opcion) {
        case 1: registrarJugador();      break;   case 2: listarJugadores();   break;
        case 3: buscarJugadorPorDni();   break;   case 4: registrarPartido(); break;
        case 5: registrarPlantel();       break;   case 6: registrarEvento();  break;
        case 7: deshacerUltimoEvento();   break;   case 8: mostrarRanking();   break;
        case 9: darDeBajaJugador();       break;   case 10: exportarReporte(); break;
        case 0: salir = true;              break;
        default: System.out.println("Opcion invalida.");
    }
}
```

4.8 Estructuras de datos

El prototipo aplica las cuatro estructuras canónicas con un caso de uso justificado en cada una:

Estructura	Uso justificado
LinkedList<EventoPartido>;	Eventos cronológicos por partido (append O(1)).
ArrayDeque (pila LIFO)	Deshacer el último evento cargado (undo).
Queue (FIFO)	Orden de sustituciones planificadas.
HashMap<Integer,T>;	Acceso por id O(1) amortizado en modo memoria.

```
// Lista enlazada: eventos cronologicos por partido (append O(1))
private final Map<Integer, LinkedList<EventoPartido>> eventosPorPartido = new HashMap<>();
RepositorioMemoria.get().eventosDe(evento.getIdPartido()).addLast(evento);

// Pila LIFO (ArrayDeque): deshacer el ultimo evento
private final Deque<EventoPartido> pilaUndo = new ArrayDeque<>();
pilaUndo.push(evento); // al registrar
EventoPartido tope = pilaUndo.pop(); // al deshacer (LIFO)

// Cola FIFO (Queue): orden de sustituciones
private final Queue<Integer> colaSustituciones = new LinkedList<>();
colaSustituciones.offer(idJugador); // encolar
Integer siguiente = colaSustituciones.poll(); // desencolar (FIFO)
```

4.9 Algoritmos de ordenación y búsqueda

Se implementan manualmente —sin Collections.sort() ni Arrays.binarySearch()— para evidenciar comprensión algorítmica. QuickSort recursivo con mediana de tres ($O(n \log n)$) esperado; la mediana de tres mitiga el peor caso ($O(n^2)$) ordena jugadores por apellido; la búsqueda binaria ($O(\log n)$) localiza por DNI sobre lista previamente ordenada.

```
public static void ordenarPorApellido(List<Jugador> j) {
    if (j == null || j.size() < 2) return;
    quickSort(j, 0, j.size() - 1);
}

private static int particionar(List<Jugador> l, int izq, int der) {
    int medio = izq + (der - izq) / 2; // mediana de tres
    if (cmp(l.get(izq), l.get(medio)) > 0) intercambiar(l, izq, medio);
    if (cmp(l.get(izq), l.get(der)) > 0) intercambiar(l, izq, der);
    if (cmp(l.get(medio), l.get(der)) > 0) intercambiar(l, medio, der);
    Jugador pivote = l.get(medio); intercambiar(l, medio, der);
    int i = izq - 1;
    for (int k = izq; k < der; k++)
        if (cmp(l.get(k), pivote) <= 0) { i++; intercambiar(l, i, k); }
    intercambiar(l, i + 1, der); return i + 1;
}

public static Jugador buscarPorDni(List<Jugador> ordenados, String dni) {
    int izq = 0, der = ordenados.size() - 1;
    while (izq <= der) {
        int medio = izq + (der - izq) / 2; // evita overflow
        int c = ordenados.get(medio).getDni().compareTo(dni);
        if (c == 0) return ordenados.get(medio); // encontrado
        if (c < 0) izq = medio + 1; else der = medio - 1;
    }
    return null;
}
```

4.10 Patrones de diseño (base AP3)

El prototipo aplica Singleton (ConexionBD), DAO (un objeto de acceso por entidad), Layered Architecture (tres capas) y Factory Method (EventoPartido.crear). En AP4 el patrón DAO se profundiza y se materializa con una interfaz (sección 5).

5. AP4 — Fase de Transición: persistencia, MySQL e interfaz

La contribución específica de AP4 convierte el prototipo en un producto desplegable con conexión real a la base de datos, atendiendo la observación de la cátedra.

5.1 Selección y justificación del patrón de diseño: DAO

Se selecciona DAO (Data Access Object) como patrón protagonista. Encapsula el acceso a datos detrás de un contrato uniforme (insertar, findByDni, findAll, darDeBaja), de modo que la capa de negocio nunca conoce JDBC ni SQL. Esto habilita el modo dual: la misma firma resuelve contra MySQL o contra memoria sin que los Service cambien. En AP4 el contrato se materializa explícitamente con la interfaz JugadorDAOInterface, que JugadorDAO implementa; la capa de negocio programa contra la abstracción (inversión de dependencias).

```
public interface JugadorDAOInterface {
    Jugador insertar(Jugador j) throws SQLException;
    Jugador findByDni(String dni) throws SQLException;
    List<Jugador> findAll() throws SQLException;
    void darDeBaja(int idJugador) throws SQLException;
}
public class JugadorDAO implements JugadorDAOInterface { ... } // impl JDBC/memoria
public class JugadorService {
    private final JugadorDAOInterface dao = new JugadorDAO(); // depende de la interfaz
}
```

Criterio	Aporte del patrón DAO
Pros	Añade la persistencia; intercambia el origen (MySQL y memoria) tras la misma interfaz; facilita el testing.
Contras	Agrega una clase/interfaz por entidad; mapeo fila a objeto repetitivo.
Soporte	Singleton (ConexionBD), Factory Method (EventoPartido.crear), Layered Architecture.

5.2 Conexión: Singleton y modo dual robusto

ConexionBD aplica Singleton: si DB_PASS está definida intenta la conexión JDBC real; si no, o si falla, cae a un repositorio en memoria informando el motivo, sin abortar.

```
private ConexionBD() {
    String url=getenv("DB_URL",URL_DEFAULT); String user=getenv("DB_USER",USER_DEFAULT);
    String pass=getenv("DB_PASS","");
    if (pass.isEmpty()) { this.modomemoria=true; return; } // demo
    try { this.connection=DriverManager.getConnection(url,user,pass); }
    catch (SQLException e) { this.modomemoria=true; } // fallback robusto
}
```

5.3 Operaciones JDBC en los DAOs

Cada DAO usa PreparedStatement (parametrizado, previene inyección SQL). Se ejemplifican las cuatro operaciones requeridas: conectar, consultar, actualizar y presentar.

```
// INSERT con clave generada
String sql="INSERT INTO jugadores (nombre,apellido,dni,fecha_nacimiento,"
    +"posicion,id_categoria,estado,fecha_alta) VALUES (?, ?, ?, ?, ?, ?, 'activo',CURDATE())";
try (PreparedStatement ps=bd.getConnection().prepareStatement(sql,RETURN_GENERATED_KEYS)){
    ps.setString(1,j.getNombre()); /* ... */ ps.executeUpdate();
    try (ResultSet rs=ps.getGeneratedKeys()){ if(rs.next()) j.setId(rs.getInt(1)); }
}
// SELECT + mapeo / UPDATE (baja logica que preserva el historial)
"SELECT * FROM jugadores WHERE dni = ?"; // -> mapear(rs)
"UPDATE jugadores SET estado='inactivo', fecha_baja=CURDATE() WHERE id_jugador = ?";
```

La carga del plantel se realiza en una transacción (setAutoCommit(false) + commit/rollback) para garantizar atomicidad.

5.4 Polimorfismo en la persistencia

EventoDAO guarda el discriminador tipo_evento y, al leer, reconstruye la subclase concreta con la fábrica EventoPartido.crear(...). Ni el DAO ni la base conocen las siete clases concretas.

5.5 Manejo de excepciones con MySQL

Las operaciones declaran SQLException; la capa de presentación captura por especificidad y, ante el fallo de conexión, el modo dual evita la caída. La escritura de reportes maneja IOException de forma diferenciada.

5.6 Utilización complementaria de arreglos y ArrayList

Se usa ArrayList (dinámico) para el ranking, cuya cantidad de elementos se desconoce y crece según los datos; y un arreglo int[] (fijo) para el resumen por tipo de evento, cuyo tamaño es la cantidad de valores del enum, con acceso O(1) por ordinal().

```
public int[] conteoGlobalPorTipo() throws SQLException {
    int[] conteo = new int[EventoPartido.Tipo.values().length]; // ARREGLO fijo
    for (Partido p : partidoDao.findAll())
        for (EventoPartido e : eventoDao.findByPartido(p.getId()))
            conteo[e.getTipo().ordinal()]++; // O(1)
    return conteo;
} // el ranking se devuelve como ArrayList<FilaRanking> (dinamico)
```

5.7 Empleo de archivos

ReporteService (CU09) exporta el ranking y el resumen a un archivo de texto con try-with-resources sobre un BufferedWriter (cierre automático) y maneja IOException, combinando el ArrayList del ranking con el arreglo int[] del resumen.

6. Trazabilidad: del caso de uso a las pruebas

Se desarrollan tres casos de uso de extremo a extremo: CU01 (autenticación), CU02 (alta de jugador) y CU07 (registro de evento).

6.1 CU01 — Iniciar sesión

Flujo: el usuario ingresa credenciales; el sistema valida el hash PBKDF2, identifica el rol y habilita el menú. Tras tres intentos fallidos, la sesión se bloquea. La presentación (MenuConsole) delega en UsuarioService.autenticar(), que recupera el usuario vía UsuarioDAO y verifica el hash con HashUtil.verificar() (comparación en tiempo constante, anti-timing).

```
// Negocio -- UsuarioService.autenticar()
public Usuario autenticar(String nombreUsuario, String pass)
    throws CredencialesInvalidasException, SQLException {
    Usuario u = dao.findByNombreUsuario(nombreUsuario);
    if (u == null || !u.isActivo()) throw new CredencialesInvalidasException();
    if (!HashUtil.verificar(pass, u.getContraseñaHash()))
        throw new CredencialesInvalidasException();
    return u;
}
```

PT01: APROBADO — acceso autorizado con rol=ENTRENADOR; menú habilitado.

6.2 CU02 — Registrar jugador

Flujo: el Secretario ingresa los datos; el sistema valida que el DNI no esté duplicado y persiste el jugador con estado activo. JugadorService.registrar() concentra las validaciones (campos obligatorios, fecha no futura, DNI válido y único) y delega en JugadorDAO.insertar().

```
// Negocio -- JugadorService.registrar()
public Jugador registrar(String nombre, String apellido, String dni,
    LocalDate fn, String posicion, int idCategoria)
    throws DatosInvalidosException, DniDuplicadoException, SQLException {
    if (nombre == null || nombre.isBlank()) throw new DatosInvalidosException("Nombre obligatorio");
    if (fn == null || fn.isAfter(LocalDate.now())) throw new DatosInvalidosException("Fecha invalida");
    ValidadorDNI.validar(dni);
    if (dao.findByDni(dni) != null) throw new DniDuplicadoException(dni); // regla de negocio
}
```

```
return dao.insertar(new Jugador(nombre.trim(), apellido.trim(), dni.trim(), fn, posicion, idCategori
a));
}
```

PT02 (alta): APROBADO — jugador persistido con id autoincremental. PT03 (DNI duplicado): APROBADO — DniDuplicadoException con mensaje claro; cero registros nuevos.

6.3 CU07 — Registrar evento

Flujo: el Entrenador selecciona partido, jugador y tipo de evento; el sistema lo registra y actualiza polimórficamente el marcador. EventoService valida la pertenencia del jugador al plantel, crea el evento con la fábrica polimórfica y recalcula el marcador iterando sobre EventoPartido.

```
// Negocio -- EventoService.registrarEvento()
public EventoPartido registrarEvento(EventoPartido.Tipo tipo, int idPartido,
                                     int idJugador, int minuto)
    throws DatosInvalidosException, SQLException {
    if (minuto < 0 || minuto > 90) throw new DatosInvalidosException("Minuto invalido (0-90)");
    boolean enPlantel = partidoDao.obtenerPlantel(idPartido).stream()
        .anyMatch(pp -> pp.getIdJugador() == idJugador);
    if (!enPlantel) throw new DatosInvalidosException("El jugador no pertenece al plantel");
    EventoPartido e = EventoPartido.crear(tipo, idPartido, idJugador, minuto); // factory
    eventoDao.insertar(e);
    int total = eventoDao.findByPartido(idPartido).stream()
        .mapToInt(EventoPartido::calcularPuntos).sum(); // polimorfico
    partidoDao.actualizarPuntos(idPartido, total, 0);
    return e;
}
```

PT08: APROBADO — secuencia TRY/CONV/TRY/PENAL/TRY: suma polimórfica $5+2+5+3+5 = 20$; ranking distribuye correctamente los puntos por jugador sin instanceof.

7. Plan de pruebas y evidencia

7.1 Resultados

ID	Caso / objeto	Tipo	Criterio	Result.
PT01	CU01 Iniciar sesión	Funcional	Acceso con rol correcto	APROBADO
PT02	CU01 (negativo)	Seguridad	Bloqueo tras 3 intentos	APROBADO
PT03	CU02 Registrar jugador	Integración	Persistido con id auto	APROBADO
PT04	CU02 (DNI duplicado)	Negocio	Rechazo con excepción	APROBADO
PT05	CU04 Dar de baja	Funcional	estado=inactive; historial	APROBADO
PT06	CU05 Registrar partido	Integración	Partido persistido	APROBADO
PT07	CU06 Registrar plantel	Funcional	Titular obligatorio	APROBADO
PT08	CU07 Registrar evento	Polimorfismo	Evento + recalcular	APROBADO
PT09	CU08 Estadísticas	Algoritmo	Ranking por puntos	APROBADO
PT10	Pila undo	Estructuras	Pop LIFO + recalcular	APROBADO
PT11	Búsqueda binaria	Algoritmo	Localiza DNI $O(\log n)$	APROBADO
PT12	QuickSort	Algoritmo	Orden por apellido	APROBADO
PT13	Conexión MySQL real	Integración	INSERT/SELECT/UPDATE	APROBADO

7.2 Casos de prueba detallados (extracto)

PT03 — CU02 DNI duplicado	
Precondición	Jugador con DNI '35211001' ya existe en el plantel.
Entrada	nombre='Pedro', apellido='Gómez', dni='35211001'.

PT03 — CU02 DNI duplicad	
Esperado	DniDuplicadoException: 'El DNI 35211001 ya se encuentra registrado'.
Obtenido	Mensaje exacto mostrado; 0 registros nuevos. APROBADO.
PT08 — CU07 Evento + rec	
Precondici	Partido id=1 con plantel cargado (#1..#5).
Entrada	TRY/Casas/12, CONV/Casas/13, TRY/Pereyra/28, PENAL/Casas/55, TRY/Ruiz/72.
Esperado	Casas 10 (1T+1C+1P), Pereyra 5, Ruiz 5; marcador local = 20.
Obtenido	Suma polim

7.3 Evidencia de ejecución (modo demo, sin MySQL)

El conjunto de pruebas se ejecuta scripteado alimentando el menú con smoke_input.txt; la corrida completa demanda menos de 2 segundos.

```
[INFO] Modo demo (en memoria). Cargando datos de prueba...
--- Padron de jugadores (ordenado QuickSort) ---
Jugador #3 - Casas, Federico [Apertura] - DNI 34567890
Jugador #4 - Herrera, Gonzalo [Ala derecho] - DNI 37124500
Jugador #1 - Pereyra, Rodrigo [Pilar] - DNI 35211001
--- CU07 Registrar evento (polimorfico) ---
OK. Evento registrado: TRY en minuto 12 (jugador #3) [5 pts]
OK. Evento registrado: CONVERSION en minuto 13 (jugador #3) [2 pts]
OK. Evento registrado: PENAL en minuto 55 (jugador #3) [3 pts]
--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---
Casas, Federico      | 1 | 1 | 1 | 0 | 10
Pereyra, Rodrigo     | 1 | 0 | 0 | 0 | 5
```

7.4 Evidencia de ejecución con conexión real a MySQL

Atendiendo la observación —«en la cuarta entrega debe ya estar con conexión a la base... algo funcional y con conexión a la base de datos»—, se ejecutó el sistema contra MySQL 8.0 y se verificó el ciclo completo. Al definir DB_PASS, el arranque ya no muestra el modo demo y opera sobre la base sistrugby_sltc.

```
Consola - App conectada a MySQL (INSERT + SELECT + polimorfismo)

PS C:\...\SistRUGBY-SLTC> $env:DB_PASS='sltc2026'
PS C:\...\SistRUGBY-SLTC> java -cp 'out;lib/mysql-connector-j-8.3.0.jar' com.sltc.sistrugby.Main

[INFO] Conectado a MySQL (modo produccion). Los datos provienen de la base sistrugby_sltc.
=====
SistRUGBY-SLTC | Santiago Lawn Tennis Club | Prototipo AP4
=====

--- Iniciar sesion ---
Usuario: entrenador1
Contraseña: *****
Bienvenido, entrenador1 [ENTRENADOR]

--- CU02 Registrar jugador ---
Nombre: Pedro  Apellido: Gomez  DNI: 40123456
Fecha nac. (YYYY-MM-DD): 2000-03-15  Posicion: Pilar  Categoria: 8
OK. Jugador creado: Jugador #6 - Gomez, Pedro [Pilar] - DNI 40123456

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---
JUGADOR      | TRY | CONV | PEN | DROP | PUNTOS
-----
Casas, Federico      | 1   | 1   | 1   | 0   | 10
Pereyra, Rodrigo     | 1   | 0   | 0   | 0   | 5
```

Figura 12 — Arranque conectado a MySQL: alta del jugador #6 (INSERT) y ranking polimórfico (SELECT).

La persistencia es real: el jugador insertado por la app se recupera con una consulta independiente desde el cliente de MySQL.

```
MySQL CLI - La fila insertada por la app esta en la base

mysql> USE sistrugby_sltc;
mysql> SELECT id_jugador, apellido, nombre, dni, estado
        -> FROM jugadores ORDER BY id_jugador;

+-----+-----+-----+-----+-----+
| id_jugador | apellido | nombre | dni      | estado |
+-----+-----+-----+-----+-----+
| 1 | Pereyra | Rodrigo | 35211001 | activo |
| 2 | Villalba | Matias | 36089452 | activo |
| 3 | Casas | Federico | 34567890 | activo |
| 4 | Herrera | Gonzalo | 37124500 | activo |
| 5 | Ruiz | Tomas | 35980012 | activo |
| 6 | Gomez | Pedro | 40123456 | activo | <-- alta persistida en MySQL
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Figura 13 — Verificación directa en MySQL: la fila (jugador #6, Gomez) quedó persistida en la tabla jugadores.

```
Persistencia en ARCHIVO: ArrayList (ranking) + arreglo int[] (resumen)

$ type reporte_sltc.txt    (archivo generado por ReporteService - CU09)

=====
SistRUGBY-SLTC - Reporte de estadísticas (CU09)
Generado: 2026-06-28 20:51:51
=====

RANKING DE JUGADORES POR PUNTOS
JUGADOR          | TRY | CONV | PEN | DROP | PUNTOS
-----
Casas, Federico  | 1   | 1    | 1   | 0    | 10
Pereyra, Rodrigo | 1   | 0    | 0   | 0    | 5

RESUMEN GLOBAL POR TIPO DE EVENTO
-----
TRY                : 2
CONVERSION         : 1
PENAL              : 1
DROP               : 0
TARJETA_AMARILLA  : 0
TARJETA_ROJA      : 0
SUSTITUCION       : 0

Fin del reporte.
```

Figura 14 — Archivo reporte_sltc.txt generado por ReporteService: ArrayList (ranking) + arreglo int[] (resumen).

Se validó además la robustez del modo dual (PT13): con el servidor detenido, el sistema informa el motivo y conmuta a memoria en lugar de abortar.

8. Atención de las observaciones previas

- AP3 (docente): «en la cuarta entrega debe ya estar con conexión a la base... funcional y con conexión a la base de datos». → AP4 completa el JDBC de todos los DAOs, unifica schema.sql con el código, entrega hashes PBKDF2 reales y aporta evidencia de ejecución conectada (sección 7.4).
- Patrón e interfaces: se explicitó el contrato del patrón DAO con la interfaz JugadorDAOInterface.
- POO: se ratifican encapsulamiento, herencia, polimorfismo y abstracción, con revisión de constructores y creación de objetos.
- Carácter incremental: este documento integra el contenido de AP1, AP2 y AP3 con las correcciones aplicadas, más la contribución de AP4.

9. Repositorio y guía de ejecución

Repositorio público: <https://github.com/nachof9/SistRUGBY-SLTC>

Modo demo (sin MySQL):

```
javac -d out -encoding UTF-8 $(find src/main/java -name "*.java")
java -cp out com.sltc.sistrugby.Main
```

Modo producción (con MySQL):

```
mysql -u root -p < sql/schema.sql
mysql -u root -p < sql/datos_iniciales.sql
set DB_PASS=tu_password
java -cp "out;lib/mysql-connector-j-8.3.0.jar" com.sltc.sistrugby.Main
```

Credenciales de prueba: admin/admin2026, entrenador1/rugby2026, secretario1/sltc2026.

10. Conclusiones

AP4 cierra el ciclo del PUD para SistRUGBY-SLTC entregando un producto funcional y con conexión real a MySQL, verificada mediante alta, consulta y actualización de registros sobre la base. El documento integra de forma incremental el análisis (AP1), el diseño y la base de datos (AP2) y el prototipo POO (AP3), e incorpora la persistencia JDBC, el patrón DAO materializado en una interfaz, el uso complementario de arreglos y ArrayList y la generación de reportes en archivos. El modo dual robusto permite ejecutar el sistema con o sin base. La arquitectura en capas y la separación que impone el DAO dejan el sistema preparado para extensiones futuras (GUI Swing, exportación a PDF/CSV, JUnit) sin reescribir la lógica existente.

11. Referencias

- Eckel, B. (2006). Thinking in Java (4.^a ed.). Prentice Hall.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- Kendall, K., & Kendall, J. (2011). Análisis y diseño de sistemas (8.^a ed.). Pearson Educación.
- Larman, C. (2003). UML y patrones (2.^a ed.). Pearson Prentice Hall.
- National Institute of Standards and Technology. (2010). NIST SP 800-132. NIST.
- Oracle Corporation. (2024). Java SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/>
- MySQL AB. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>